

Class Note: File Upload in Node.js Using Formidable (Without Express)



Drag&Drop files here

or

[Browse Files](#)

Learning Objectives

At the end of this lesson, students should be able to:

1. Understand how file uploads work in a Node.js server.
2. Use the **Formidable module** to handle single and multiple file uploads.
3. Manage uploaded files — rename, move, and validate them.
4. Implement error handling for invalid or oversized files.
5. Build an upload system progressively — from a simple example to an advanced file manager.



Background Concept



What Is File Upload?

File upload allows users to send files (images, documents, videos, etc.) from their computer to a server.

This process involves:

- **Frontend:** An HTML form using `enctype="multipart/form-data"`.
- **Backend:** A Node.js server that receives and processes the file.



Why Use Formidable?

Formidable is a Node.js library for parsing form data, especially **multipart forms** used in file uploads. It:

- Reads uploaded files
- Saves them temporarily
- Provides file details (name, size, type, path)
- Helps developers easily move or rename uploaded files



Install Formidable

```
npm install formidable
```



Progressive Examples

We'll build from the **simplest** working version to **complex** upload systems. Each step adds a new capability.



Example 1 — Hello File Upload (Single File)



Objective: Handle a simple upload of one file.

Code:

```
const http = require('http');
const formidable = require('formidable');
const fs = require('fs');

http.createServer((req, res) => {
  if (req.url === '/upload' && req.method.toLowerCase() === 'post') {
    const form = new formidable.IncomingForm();

    form.parse(req, (err, fields, files) => {
      if (err) {
        res.writeHead(500, { 'Content-Type': 'text/plain' });
        res.end('File upload failed!');
        return;
      }

      // Display file info
      const file = files.fileupload[0];
      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.write(`<h3>File uploaded successfully!</h3>`);
      res.write(`<p>Original Filename: ${file.originalFilename}</p>`);
      res.write(`<p>Temporary Path: ${file.filepath}</p>`);
      res.end();
    });
  } else {
    // HTML upload form
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write(`
      <form action="/upload" method="post" enctype="multipart/form-data">
        <input type="file" name="fileupload"><br><br>
        <input type="submit" value="Upload File">
      </form>
    `);
    res.end();
  }
}).listen(8080);

console.log("Server running at http://localhost:8080");
```

Explanation:

- The form uses `enctype="multipart/form-data"`.
- `formidable.IncomingForm()` parses the upload.
- `files.fileupload[0]` gives access to the uploaded file.

- The file is stored in a temporary system folder automatically.

Example 2 — Move File to “uploads” Folder

 **Objective:** Save the uploaded file into a permanent folder.

Code:

```
const http = require('http');
const formidable = require('formidable');
const fs = require('fs');

http.createServer((req, res) => {
  if (req.url === '/upload' && req.method.toLowerCase() === 'post') {
    const form = new formidable.IncomingForm();
    form.uploadDir = './uploads'; // Folder to store files
    form.keepExtensions = true; // Keep original file extensions

    form.parse(req, (err, fields, files) => {
      if (err) {
        res.writeHead(500, { 'Content-Type': 'text/plain' });
        res.end('Upload error!');
        return;
      }

      const file = files.fileupload[0];
      const oldPath = file.filepath;
      const newPath = form.uploadDir + '/' + file.originalFilename;

      fs.rename(oldPath, newPath, (err) => {
        if (err) throw err;
        res.writeHead(200, { 'Content-Type': 'text/html' });
        res.write(`<h3>File uploaded and moved!</h3>`);
        res.write(`<p>Saved as: ${file.originalFilename}</p>`);
        res.end();
      });
    });
  } else {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write(`
      <form action="/upload" method="post" enctype="multipart/form-data">
        <input type="file" name="fileupload"><br><br>
        <input type="submit" value="Upload File">
      </form>
    `);
    res.end();
  }
}).listen(8080);

console.log("Server running at http://localhost:8080");
```

 Explanation:

- `uploadDir` defines where files will be stored.
- `keepExtensions` keeps the `.jpg` , `.png` , etc.
- `fs.rename()` moves file from temp to `/uploads` folder.

 Example 3 — Multiple File Uploads

 **Objective:** Allow uploading multiple files at once.

Code:

```
const http = require('http');
const formidable = require('formidable');
const fs = require('fs');

http.createServer((req, res) => {
  if (req.url === '/upload' && req.method.toLowerCase() === 'post') {
    const form = new formidable.IncomingForm();
    form.uploadDir = './uploads';
    form.keepExtensions = true;
    form.multiples = true; // Enable multiple file upload

    form.parse(req, (err, fields, files) => {
      if (err) throw err;

      files.fileupload.forEach(file => {
        const oldPath = file.filepath;
        const newPath = form.uploadDir + '/' + file.originalFilename;
        fs.renameSync(oldPath, newPath);
      });

      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.write('<h3>All files uploaded successfully!</h3>');
      res.end();
    });
  } else {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write(`
      <form action="/upload" method="post" enctype="multipart/form-data">
        <input type="file" name="fileupload" multiple><br><br>
        <input type="submit" value="Upload Files">
      </form>
    `);
    res.end();
  }
}).listen(8080);

console.log("Server running at http://localhost:8080");
```

Explanation:

- `multiple` in HTML allows selecting more than one file.
- `form.multiples = true` enables it in the backend.
- Loop through `files.fileupload` to handle each file.

Example 4 — Rename Files with Timestamp

 **Objective:** Avoid overwriting files with the same name.

Code (building on Example 3):

```
const newName = Date.now() + '_' + file.originalFilename;  
const newPath = form.uploadDir + '/' + newName;
```

Replace the renaming section in the loop with this.

 **Result:**

If you upload `photo.jpg`, it becomes something like `1730742659132_photo.jpg`.

 **Why:**

This ensures unique filenames — useful in multi-user systems.

Example 5 — Restrict File Types (Allow Images Only)

 **Objective:** Prevent users from uploading non-image files.

Code:

```
const http = require('http');
const formidable = require('formidable');
const fs = require('fs');

http.createServer((req, res) => {
  if (req.url === '/upload' && req.method.toLowerCase() === 'post') {
    const form = new formidable.IncomingForm();
    form.uploadDir = './uploads';
    form.keepExtensions = true;

    form.onPart = function (part) {
      if (!part.filename || part.mimetype.startsWith('image/')) {
        form.handlePart(part);
      } else {
        console.log('Rejected non-image file:', part.filename);
      }
    };

    form.parse(req, (err, fields, files) => {
      if (err) throw err;
      files.fileupload.forEach(file => {
        const oldPath = file.filepath;
        const newPath = './uploads/' + file.originalFilename;
        fs.renameSync(oldPath, newPath);
      });

      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.write('<h3>Only image files were accepted and uploaded.</h3>');
      res.end();
    });
  } else {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write(`
      <form action="/upload" method="post" enctype="multipart/form-data">
        <input type="file" name="fileupload" multiple><br><br>
        <input type="submit" value="Upload Files (Images Only)">
      </form>
    `);
    res.end();
  }
}).listen(8080);

console.log("Server running at http://localhost:8080");
```

Explanation:

- The `onPart()` method runs for each uploaded part.
- Only files starting with `image/` MIME type are accepted.

Example 6 — Set Maximum File Size

 **Objective:** Limit upload size to prevent large files.

Code:

```
const form = new formidable.IncomingForm();
form.maxFileSize = 5 * 1024 * 1024; // 5MB
```

Full Example:

```
form.parse(req, (err, fields, files) => {
  if (err) {
    res.writeHead(413, { 'Content-Type': 'text/plain' });
    res.end('File too large! Maximum size is 5MB.');
```

```
    return;
  }
  // Handle upload as usual
});
```

Note:

Formidable automatically stops uploads exceeding this size.

Example 7 — Organize Uploads into User Folders

 **Objective:** Create personalized folders dynamically.

Code:

```
const userId = 'user123';
const dir = './uploads/' + userId;

// Create folder if not exists
if (!fs.existsSync(dir)) {
  fs.mkdirSync(dir, { recursive: true });
}

const newPath = dir + '/' + Date.now() + '_' + file.originalFilename;
fs.renameSync(file.filepath, newPath);
```

Real-world Use:

In an app like **Swift Space**, each agent's uploads can be saved under their ID folder.

Example 8 — Display Uploaded File Information

 **Objective:** Show uploaded file details to the user.

```
res.write('<h3>File Upload Summary:</h3>');
res.write('<ul>');
files.fileupload.forEach(file => {
  res.write('<li>${file.originalFilename} - ${(file.size / 1024).toFixed(2)} KB</li>');
});
res.write('</ul>');
res.end();
```

Result:

After upload, users see a neat list of uploaded filenames and sizes.

Example 9 — Handling Errors Gracefully

 **Objective:** Show meaningful messages when something goes wrong.

```
form.on('error', (err) => {
  console.log("Error occurred:", err.message);
  res.writeHead(500, { 'Content-Type': 'text/plain' });
  res.end('File upload failed: ' + err.message);
});
```

 Purpose:

Prevents server crashes and helps debugging.

 Summary Table

Feature	Code/Property	Description
Upload Directory	form.uploadDir	Defines where files are stored
Keep Extension	form.keepExtensions	Keeps original file extensions
Multiple Files	form.multiples = true	Allows multiple uploads
File Size Limit	form.maxFileSize	Restricts file size
File Filtering	form.onPart()	Filters allowed types
Rename File	fs.rename()	Moves or renames uploaded files
Folder Creation	fs.mkdirSync()	Creates folders dynamically

 Real-Life Scenario

In real-world platforms:

- **Real estate apps:** Upload property photos.
- **Learning apps:** Upload assignments and PDFs.
- **Social media:** Upload profile and cover images.

Formidable provides a lightweight, flexible way to implement all these.



Quick Review Questions

1. What does `enctype="multipart/form-data"` do in a form?
2. Why is `form.keepExtensions = true` useful?
3. How can you allow multiple files in one upload?
4. Write a snippet that renames uploaded files with timestamps.
5. What is the role of `fs.rename()` in file upload?